

Slide Deck: AI Engineer Assessment - Python Q&A Assistant

Slide 1: Title

Python Programming Q&A Assistant

Analytics Vidhya AI Engineer Assessment

Candidate: Nikhil Kumar

Slide 2: Problem Statement

Goal: Build an AI-powered Python Q&A system for data science learners.

Requirements:

- Grounded answers using Stack Overflow data.
- Publicly accessible REST API with a User Interface.
- FastAPI backend framework.
- Robust RAG pipeline.

Slide 3: Architecture Overview

Free-Tier Optimized Cloud Setup:

- **Frontend UI:** Streamlit (Chat Interface)
- **Frontend/Proxy:** Nginx (Reverse proxy & timeouts)
- **API Backend:** FastAPI running via Uvicorn/Gunicorn
- **Vector Database:** Pinecone Cloud (Serverless, Free Tier)
- **LLM Engine:** AWS Bedrock (Qwen 3 Coder Next)
- **Embeddings:** AWS Bedrock (Amazon Titan Text v2)
- **Host:** AWS EC2 `t2.micro`

Slide 4: Architecture Diagram

```
graph LR
    A[User] -->|Interacts| F[Streamlit UI]
    F -->|POST /ask| B[Nginx Reverse Proxy]
    B --> C[FastAPI Server]
    C -->|1. Query| D[Pinecone Vector Store]
    D -->|2. Context| C
    C -->|3. Prompt| E[AWS Bedrock LLM]
    E -->|4. Answer| C
    C -->|5. Response| F
```

Slide 5: Data Ingestion Pipeline

Processing the Kaggle Dataset:

1. **Merge & Limit:** Combine `Questions.csv` , `Answers.csv` , and `Tags.csv` . Filter to the top 1,000 highest scoring QA pairs to maximize quality.
2. **Chunking:** LangChain's `RecursiveCharacterTextSplitter` (Size 1000, Overlap 200).
3. **Embedding:** AWS Titan (1024 dimensions).
4. **Parallel Upsert:** Highly optimized parallel batch uploads to Pinecone using `ThreadPoolExecutor` and `tqdm` for real-time progress tracking.

Slide 6: Key Design Decisions

- **Streamlit Integration:** Gives users a tangible, "ChatGPT-like" experience over a raw API.
- **Cloud Vector DB over Local FAISS:** Saves memory on the EC2 instance, enabling the use of a `t2.micro` (1GB RAM).
- **Docker Compose limits:** Finely tuned memory constraints across the 3 containers (`app`, `nginx`, `streamlit`) to prevent Out-Of-Memory (OOM) errors.

Slide 7: RAG Prompting Strategy

System Prompt:

- Strict adherence to provided context.
- Fallback mechanism for insufficient context.
- Enforced Markdown formatting for code snippets.

Output Parser: `StrOutputParser` extracts the raw string, returned in a structured Pydantic response model.

Slide 8: Scaling Strategy (100+ Concurrent Users)

Current Bottlenecks: `t2.micro` CPU/RAM, Bedrock Rate Limits.

Scaling Solutions:

1. **Compute:** Upgrade EC2 instance size or migrate to AWS ECS (Fargate) for auto-scaling containers.
2. **Async Bedrock Calls:** Utilize asynchronous Bedrock invocation (`ainvoke`) in LangChain to unblock the event loop.
3. **Caching:** Implement **Redis** (ElastiCache) to cache exact/similar queries to reduce LLM calls and latency.

Slide 9: Cost Analysis (Production Scale)

- **API Gateway + Application Load Balancer:** ~\$20/mo
- **Compute (ECS Fargate):** ~\$30 - \$50/mo based on traffic.
- **Vector DB (Pinecone Standard):** ~\$70/mo for larger indexes.
- **LLM (AWS Bedrock):** Pay-per-token. Caching significantly reduces this cost.

Slide 10: Conclusion & Next Steps

Summary: We successfully built a highly optimized, full-stack RAG application with an interactive UI.

Next Steps:

- Implement semantic caching.
- Add user feedback endpoints (thumbs up/down) in Streamlit to fine-tune retrieval.
- Expand dataset to include official Python documentation.